EXPERIMENT-1

Aim:

- Solving 2 water jug Problem: Write a program to solve the 2-water jug problem by manually providing moves from the user.
- Write a program to implement Missionaries and Cannibals problem.
  Constraints: three missionaries and three cannibals must cross a river using a boat which can carry at most two people, under the constraint that, for both banks, if there are missionaries present on the bank, they cannot be outnumbered by cannibals (if they were, the cannibals would eat the missionaries). The boat cannot cross the river by itself with no people on board.

Theory:

The water jug problem and the Missionaries and Cannibals problem are classical examples used in Artificial Intelligence to understand state-space search and problem solving strategies.

The two-water jug problem consists of two jugs with fixed capacities and a target amount of water to be measured. The sequence of some valid operations like filling a jug, emptying a jug, transferring the water from one jug to another solves the problem. In the current experiment, we are providing moves manually and it shows clearly how each action changes the state successfully. Through this technique, the concept of state transitions becomes much clearer, and by instructing the agent to reach a goal state, we can show it how to do so in a step by step manner from the initial state.

The Missionaries and Cannibals problem is a constraint-based river crossing problem where safety rules are to be strictly followed. The mission is to cross the river by transferring all the cannibals and missionaries to the other bank. The boat transports two people at a time. It is invalid if at any bank the cannibals outnumber the missionaries. The missionaries can never be in an invalid state as they will be eaten by cannibals otherwise. This problem shows the importance of checking for the validity of the state while checking the possible moves, and restraining from unsafe or already repeated states.

Algorithm:

Two Water Jug Problem:-

1. Start.
2. Define functions for all valid water transfers between the two jugs.
3. Initialise a list of two elements with zero.
4. Take input from user regarding which water transfer operation to perform and print the current list showing the state of both the jugs.
5. Continue taking inputs from until goal state is reached.
6. Stop.

Missionaries and Cannibals Problem:-

1. Start.
2. Call missionaries_cannibals() function.
3. Initialise start and end states along with all valid moves/transfers.
4. Create a queue for all valid states and a set for visited states.
5. Push initial state in the queue and visited set.
6. Enter while loop, which terminates upon reaching the end_state.
7. In the loop, generate new_states and check whether it is valid by passing it to the is_valid function.
8. If valid, append it into the queue and visited set.
9. On reaching, the goal state, the path is printed and the control exits the function.
10. If end_state is not reached, the while loop is exited and “No solution found” is printed.
11. Stop.

Dataset Description:

Source: User defined input

Attributes: Jug capacities, water levels, missionaries, cannibals, boat position

Size: small finite state space

Implementation:

Two Water Jug Problem:-

```
# Function to fill Jug X completely

def fillX(l, capX, capY):

    print("You have called Fill X")

    l[0] = capX

    return l


# Function to fill Jug Y completely

def fillY(l, capX, capY):

    print("You have called Fill Y")

    l[1] = capY

    return l


# Function to empty Jug X

def emptyX(l, capX, capY):

    print("you have called Empty X")

    l[0] = 0

    return l


# Function to empty Jug Y

def emptyY(l, capX, capY):

    print("you have called Empty Y")

    l[1] = 0

    return l


# Function to pour water from Jug X into Jug Y
```

```
def pourX(l, capX, capY):
    print("you have called Pour X into Y")
    space = capY - l[1]          # Calculate available space in Jug Y
    pour = min(space, l[0])
    l[0] = l[0] - pour
    l[1] = l[1] + pour
    return l

# Function to pour water from Jug Y into Jug X
def pourY(l, capX, capY):
    # Water is transferred from Jug Y to Jug X until Jug X is full
    # or Jug Y becomes empty
    print("you have called Pour Y into X")
    space = capX - l[0]          # Calculate available space in Jug X
    pour = min(space, l[1])
    l[0] = l[0] + pour
    l[1] = l[1] - pour
    return l

# function to solve the Two Water Jug Problem using user input
def twoWaterJug(capX, capY, target):
    # Initial state: both jugs are empty
    l = [0, 0]

    # Keep taking user input until exit option is chosen
    while True:
        ch = int(input(
            "Enter 1. Fill X\n"
```

```
    "2. Fill Y\n"
    "3. Empty X\n"
    "4. Empty Y\n"
    "5. Pour X into Y\n"
    "6. Pour Y into X\n"
    "7. Exit\n"
))

if ch == 1:
    l = fillX(l, capX, capY)
    print(l)
elif ch == 2:
    l = fillY(l, capX, capY)
    print(l)
elif ch == 3:
    l = emptyX(l, capX, capY)
    print(l)
elif ch == 4:
    l = emptyY(l, capX, capY)
    print(l)
elif ch == 5:
    l = pourX(l, capX, capY)
    print(l)
elif ch == 6:
    l = pourY(l, capX, capY)
    print(l)
elif ch == 7:
    break
```

```
# Call the function with Jug X capacity = 3, Jug Y capacity = 5, target = 4
twoWaterJug(3, 5, 4)
print("Goal state reached")
```

Missionaries and Cannibals Problem:-

```
from collections import deque


def is_valid(m,c,boat):           #checking validity of the state
  return (m==0 or m>=c) and (3-m == 0 or 3-m >= 3-c)


def missionaries_cannibals():
  start_state= (3,3,0)
  end_state=(0,0,1)               #initializing start state and end state
  transfers= [(2,0), (0,2), (1,1), (1,0),(0,1)]     #valid boat moves
  queue=deque()
  visited=set()
  queue.append((start_state,[start_state]))  #append start_state in queue
  visited.add(start_state)             #and in visited set


  while queue:
    (m,c,boat), track=queue.popleft()
    if (m,c,boat)==end_state:            #check if end_state reached
      print("Solution found: ")
      for state in track:                #print the track/states
        print(state)
      return
    else:
```

```
      for a,b in transfers:           #else generate new_state

        if (boat==0):

          new_state=(m-a,c-b,1)

        else:

          new_state=(m+a,c+b,0)


        if is_valid(new_state[0], new_state[1], new_state[2]) and new_state
not in visited:                         #check validity before appending

          queue.append((new_state, track+[new_state]))

          visited.add(new_state)

  print("No solution found!")           #print if solution not found

missionaries_cannibals()
```

Output:

Two Water Jug Problem:-

```
•••  Enter 1. Fill X
     2. Fill Y
     3.Empty X
     4.Empty Y
     5.Pour X into Y
     6.Pour Y into X
     7.Exit
     2
     You have called Fill Y
     [0, 5]
     Enter 1. Fill X
     2. Fill Y
     3.Empty X
     4.Empty Y
     5.Pour X into Y
     6.Pour Y into X
     7.Exit
     6
     you have called Pour Y into X
     [3, 2]
```

```
Enter 1. Fill X
2. Fill Y
3.Empty X
4.Empty Y
5.Pour X into Y
6.Pour Y into X
7.Exit
3
you have called Empty X
[0, 2]
Enter 1. Fill X
2. Fill Y
3.Empty X
4.Empty Y
5.Pour X into Y
6.Pour Y into X
7.Exit
6
you have called Pour Y into X
[2, 0]
```

```
Enter 1. Fill X
2. Fill Y
3.Empty X
4.Empty Y
5.Pour X into Y
6.Pour Y into X
7.Exit
2
You have called Fill Y
[2, 5]
Enter 1. Fill X
2. Fill Y
3.Empty X
4.Empty Y
5.Pour X into Y
6.Pour Y into X
7.Exit
6
you have called Pour Y into X
[3, 4]
```

```
Enter 1. Fill X
2. Fill Y
3.Empty X
4.Empty Y
5.Pour X into Y
6.Pour Y into X
7.Exit
3
you have called Empty X
[0, 4]
Enter 1. Fill X
2. Fill Y
3.Empty X
4.Empty Y
5.Pour X into Y
6.Pour Y into X
7.Exit
7
Goal state reached
```

Missionaries and Cannibals Problem:-

```
Solution found:
(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)
```

Result Analysis:

For Two Water Jug Problem, we can observe that the code lets the user to try and solve the problem by giving manual inputs. As each time, an operation is chosen by the user, corresponding arithmetic changes take place in the list that contains the state of the two jugs. The user exits as the goal state is reached.

In the Missionaries and Cannibals Problem, we have in a way automated the task. The states are generated automatically in the code and are checked for validity. All valid moves are stored in the queue and the entire path is printed once the end state is reached.

Conclusion:

Together, these experiments illustrate important AI concepts such as state representation, valid actions, constraints, and goal testing. They provide a strong foundation for understanding more advanced search algorithms used in intelligent systems.